SignVaani ISL Dataset & Data Collection Guide

Dataset sources, collection protocol, repository integration, and documentation templates

---

Prepared for the SignVaani repository\ Version 1.0 | October 2026

Contents

1 Purpose of this document 1
2 Dataset options 1
2.1 INCLUDE: the best first dataset for this repository 1
2.2 iSign and CISLR: useful for future continuous translation 1
2.3 ISL-CSLTR: optional sentence-level resource 1
2.4 Custom hospital dataset: required for the final project 1
3 Recommended project strategy 2
4 Dataset format for SignVaani 2
5 Recording protocol 2
5.1 Before recording 2
5.2 During recording 3
5.3 Quality checks 3
6 Keypoint extraction and conversion 3
7 Training and evaluation 3
8 Dataset datasheet template 4
9 Privacy, ethics and licensing 4
10 Final checklist 5
11 Sources 5

1 Purpose of this document

This guide explains where to obtain Indian Sign Language (ISL) data, which datasets fit the current SignVaani codebase, how to prepare data for training, and how to document a new dataset responsibly. It is written for the repository at github.com/kartheekvineela-alt/ISL.

The most important recommendation is to separate two goals. Public datasets are useful for testing the pipeline and pre-training. A small, consented dataset recorded for the chosen hospital vocabulary is still needed for a credible final result on unseen signers.

"Do not claim that a model works well merely because it achieves high accuracy on a random split. The final test set should contain signers who were not present in training."

2 Dataset options

2.1 INCLUDE: the best first dataset for this repository

INCLUDE is an isolated-sign dataset released by AI4Bharat. Its dataset card lists 263 labels and approximately 4,292 videos, covering categories such as people, places, animals, colours, jobs, days and society. The videos can be downloaded from Zenodo, and the official AI4Bharat repository includes MediaPipe keypoint generation and training scripts.

INCLUDE matches SignVaani's current one-clip/one-label classifier much better than a sentence-translation corpus. It is appropriate for validating landmark extraction, augmentation, model training and ONNX inference. Its main limitation is domain and signer mismatch: the dataset documentation notes that the recordings come from Chennai and do not represent every regional ISL variant.

2.2 iSign and CISLR: useful for future continuous translation

The iSign benchmark provides resources for ISL video-to-English translation, pose-to-English translation, isolated word recognition and related tasks. It also links to CISLR, a large word-level corpus. These resources are valuable for research, but their continuous-translation tasks do not directly fit the current SignVaani folder format, which expects isolated clips inside a sign directory.

Use iSign/CISLR after the isolated-sign prototype is stable, or when building a separate continuous-translation pipeline. Do not mix sentence-level videos into the current classifier without creating correct clip-level labels.

2.3 ISL-CSLTR: optional sentence-level resource

The ISL-CSLTR listing is intended for continuous sign-language translation and recognition. Treat it as an exploratory source. Check its licence, annotation quality and access terms before redistribution, and use it for sentence-level experiments rather than as a drop-in replacement for INCLUDE.

2.4 Custom hospital dataset: required for the final project

The repository's current vocabulary file defines a hospital domain with signs such as `ME`, `YOU`, `DOCTOR`, `PAIN`, `FEVER`, `MEDICINE`, `HELP`, `TOILET`, `NEED`, `WHERE`, `YES`, `NO` and `NOT`, plus `IDLE`. Record the signs that you can have checked by an ISL user or interpreter. The blueprint target is at least 6 signers and 20–30 clips per sign; the minimum acceptable pilot can be smaller, but the report must state the actual numbers.

3 Recommended project strategy

1. Use INCLUDE to validate the software pipeline and establish an initial baseline.
2. Record a custom hospital dataset with consent and signer IDs.
3. Fine-tune or retrain on the custom data.
4. Reserve one or more complete signers for testing.
5. Report accuracy, macro F1, confusion pairs, latency and false triggers.

This approach is more defensible than claiming that a model trained only on public videos will generalise to webcam users from a different region.

4 Dataset format for SignVaani

The repository expects one NumPy file per clip:

```
data/keypoints/<SIGN>/<SIGNER>_<TAKE>.npy
```

Each file should contain a float32 array with shape `(T, 63, 3)`. The 63 points are 9 selected pose points, 12 selected face points, 21 left-hand points and 21 right-hand points. Missing detections may be stored as `NaN`; `signvaani.landmarks.preprocess` interpolates short gaps, resamples the clip to 32 frames, normalizes it by shoulder centre and width, and appends velocity features to produce `(32, 378)` model input.

A correct custom dataset layout is:

```
data/keypoints/
  IDLE/
    s01_01.npy
    s01_02.npy
  PAIN/
    s01_01.npy
    s02_01.npy
  DOCTOR/
    s01_01.npy
    s02_01.npy
```

The signer ID must be the first part of the filename before the underscore. This is how `signvaani.data.index_clips` builds signer-independent splits.

5 Recording protocol

5.1 Before recording

Choose a fixed vocabulary and assign one canonical label to each sign. Verify each sign with an ISL user, interpreter or a trusted ISL dictionary. Record the regional variant used. Prepare a quiet room, a webcam at approximately upper-body height, stable lighting and enough space for both hands and face.

Use anonymous IDs such as `s01`, `s02` and `s03`. Do not put names, phone numbers or other personal information in filenames. Obtain written consent before recording. Explain the purpose, storage method, possible publication and the option to withdraw.

5.2 During recording

Record around 2–3 seconds per clip at approximately 30 FPS. Ask the signer to begin from a neutral position, perform one sign naturally, and return to rest. Vary distance, background, lighting and clothing across takes, but keep the camera view usable. Record `IDLE` separately with hands resting and with natural non-sign movements such as adjusting hair or looking around.

A practical target is 5 clips per sign per signer during the first pilot, followed by additional clips for confusing signs. For a final 40-class system with 6 signers and 20 clips per class, the dataset would contain approximately 4,800 clips; plan storage and review accordingly.

5.3 Quality checks

After every session, check that the clip contains the complete sign, both hands are visible when required, the label is correct, and the signer returns to neutral. Reject clips with severe tracking failure, accidental multiple signs, camera obstruction or an incorrect label. Keep a simple log of rejected clips and the reason.

6 Keypoint extraction and conversion

The official INCLUDE code demonstrates keypoint generation from videos. For SignVaani, the final saved representation should be the 63-point raw landmark sequence rather than the already flattened 378-feature sequence, because the repository's preprocessing code performs normalization and resampling itself.

A conversion script should follow this sequence:

1. Open one source video with OpenCV.
2. Run MediaPipe Holistic on each frame.
3. Call the same point-selection logic as `signvaani.landmarks.frame_keypoints`.
4. Stack frames into `(T, 63, 3)` and save as float32 `.npy`.
5. Write the file under `data/keypoints/<SIGN>/<SIGNER>_<TAKE>.npy`.
6. Run a shape and label audit before training.

Do not silently mirror two-handed signs. Mirroring should only be used after checking that the sign remains valid and equivalent for the target ISL variant.

7 Training and evaluation

Start with a pipeline check using synthetic data, but never report synthetic accuracy as project performance:

```
python -m signvaani.make_synthetic --out data/synthetic
python -m signvaani.train --data data/synthetic --random-split
```

For a real custom dataset, use signer-independent evaluation:

```
python -m signvaani.train \
  --data data/keypoints \
  --test-signers s05 \
  --val-signers s04 \
  --out models/best.pt
```

```
python -m signvaani.evaluate \
  --ckpt models/best.pt \
  --signers s05
```

For leave-one-signer-out evaluation, use:

```
python -m signvaani.train --data data/keypoints --loso
```

The report should include signer-dependent results only as a secondary reference. The primary result should be the unseen-signer result. Include top-1 accuracy, top-3 accuracy, macro F1, per-class recall, a confusion matrix and the most confused sign pairs. For the live application, also measure end-to-end latency, FPS, missed signs and false triggers during idle periods.

8 Dataset datasheet template

Complete this table after data collection. Do not leave placeholders in the final report.

Field	Value to record
Domain	Hospital communication or chosen domain
Classes	Number of signs plus `IDLE`; list the labels and label definitions
Signers	Anonymous IDs and total number; do not include names
Clips	Takes per class per signer and total clips
Format	`data/keypoints/<SIGN>/<signer>_<take>.npy`, float32 `(T,63,3)`
Points	9 pose + 12 face + 21 left hand + 21 right hand
Frame rate	Approximate capture FPS and duration per clip
Sign source	Dictionary, interpreter or ISL user who checked labels
Recording conditions	Camera, distance, lighting, backgrounds and locations
Consent	Where consent records are stored and who may access them
Raw video	Whether raw video is retained privately or deleted
Licence	Only state a public licence if every signer agreed to release
Limitations	Regional variants, class imbalance, tracking failures and other known limits

9 Privacy, ethics and licensing

Store raw video privately unless participants explicitly consent to publication. A keypoint file can still carry information about movement and identity, so it should not be treated as automatically anonymous. Use access controls, remove identifying metadata, and document who can use the data.

Do not redistribute downloaded public datasets under a new licence unless the original terms allow it. Cite INCLUDE, iSign, CISLR or ISL-CSLTR according to their official instructions. For the custom dataset,

choose a licence only after consent has explicitly covered public release; otherwise publish the datasheet and code while keeping the data private.

The tool is an assistive prototype and should not replace a qualified interpreter in medical, legal or emergency settings. Have an ISL user or interpreter review the labels, sentence rules and spoken output before presenting accuracy claims.

10 Final checklist

1. ☐ Vocabulary labels are fixed and verified by an ISL user or interpreter.
2. ☐ At least one `IDLE` recording session is included.
3. ☐ Signer IDs are anonymous and filenames follow the required format.
4. ☐ Every `.npy` file has shape `(T, 63, 3)` and dtype `float32`.
5. ☐ The test signer never appears in the training set.
6. ☐ `models/best.pt` and the exported ONNX model are generated from documented commands.
7. ☐ Accuracy, macro F1, confusion matrix and latency are measured on real data.
8. ☐ Consent, storage, licence and known limitations are documented.
9. ☐ Hindi output is checked by a native speaker; English output is checked for grammar.
10. ☐ The final README no longer contains `[XX]` result placeholders.

11 Sources

AI4Bharat INCLUDE dataset card\ AI4Bharat INCLUDE code repository\ INCLUDE Zenodo record\ iSign project and dataset links\ ISL-CSLTR dataset listing\ SignVaani repository\

This guide is a project document, not a substitute for the dataset creators' licences, consent forms, or official documentation. Check the source page before downloading or publishing data.